

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	M. Meghana
Name	192525435

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I M. Meghana(192525435) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M. Meghana*

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1. HDFS-Based Storage Solution for a Media Platform

Introduction

HDFS stands for Hadoop Distributed File System. It is the storage system of Hadoop. It is mainly used for storing very large amounts of data by dividing the data into blocks and storing those blocks on different computers called Data Nodes.

For a media platform, a huge amount of data is generated every day. This may include videos, images, audio files, user details, viewing history, comments, and log files. Therefore, the storage system should be highly available, fault tolerant, scalable and cost efficient.

The given requirements are:

1. Data availability should be more than 99.9%.
2. Replication factor should not be more than 3.
3. Storage growth should be cost efficient.

HDFS Storage Design

In HDFS, there are two main types of nodes:

- Name Node
- Data Node

The Name Node manages the metadata and information about files. Data Nodes actually store the data blocks.

For a media platform, multiple Data Nodes can be used to store large media files.

1. Name Node High Availability

Normally, the Name Node is a very important component because it manages the HDFS file system. If the Name Node fails, users may not be able to access the stored files.

To solve this problem, Name Node High Availability can be used.

There will be:

- Active Name Node
- Standby Name Node

The Active Name Node performs normal operations. The Standby Name Node keeps a copy of the required metadata and is ready to take over if the Active Name Node fails.

Therefore, Name Node failure will not cause complete system failure.

2. Replication Factor

HDFS provides fault tolerance using data replication.

For this problem, the replication factor should be set to **3**, because the maximum allowed value is 3.

For example, if a video is divided into blocks, one block can have three copies stored on different Data Nodes.

If one Data Node fails, the same block can be obtained from another Data Node.

Replication therefore improves:

- Data availability
- Fault tolerance
- Reliability

However, replication also requires additional storage. Therefore, using a replication factor greater than 3 is not allowed and is unnecessary for this requirement.

3. Rack Awareness

HDFS should also use rack awareness.

A rack is a group of physical servers. If all three copies of a block are stored in the same rack and that rack fails, all copies may become unavailable.

Therefore, HDFS distributes replicas across different racks whenever possible.

This protects the system from both server failure and rack failure.

4. Cost-Efficient Storage Growth

Media data increases continuously because new videos and other files are added every day.

HDFS supports horizontal scaling. Instead of replacing the complete storage system, new Data Nodes can be added when more storage is required.

For example:

Initial cluster → Add DataNodes → Increase storage capacity

This is more cost efficient than purchasing a very large storage system at the beginning.

The organization can also use:

- High-capacity hard disks for large media files
- Faster storage for frequently accessed data

- Data compression where suitable
- Storage policies for old and rarely accessed files

5. Availability

The availability requirement is more than 99.9%.

This can be supported by:

- Name Node High Availability
- Replication factor 3
- Multiple Data Nodes
- Rack awareness
- Automatic block recovery
- Failure detection

If one Data Node fails, the data can still be read from another replica.

Advantages

The proposed HDFS design provides:

1. High availability
2. Fault tolerance
3. Large-scale storage
4. Easy expansion
5. Cost-efficient growth
6. Protection against Data Node failure
7. Protection against rack failure

2.A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

MapReduce Workflow for Log Data Within 20 Minutes

Introduction

MapReduce is a programming model used in Hadoop for processing large amounts of data. It divides a large task into smaller tasks and executes them in parallel.

In this problem, the system has to process log data within a 20-minute batch window. The number of cluster nodes is also limited.

Therefore, the MapReduce job must be designed carefully so that available resources are used efficiently.

Log Data

Log data may contain information such as:

- User ID
- IP address
- URL
- Date and time
- HTTP status code
- Number of requests
- Error information

For example:

192.168.1.10 GET /home 200

192.168.1.11 GET /login 200

192.168.1.12 GET /home 404

The system may need to find the number of requests for each URL or count errors.

MapReduce Process

MapReduce mainly consists of:

1. Input
2. Map
3. Shuffle and Sort

4. Reduce

5. Output

1. Input Phase

The log files are stored in HDFS.

HDFS divides large files into blocks. These blocks can be processed independently.

Different mapper tasks can process these blocks.

2. Map Phase

The Mapper reads each log record and generates key-value pairs.

The mapper does not produce the final answer. It produces intermediate results.

3. Combiner

A Combiner can be used before sending data to the reducers.

This reduces the amount of data transferred over the network.

This is especially useful because the number of cluster nodes is limited.

4. Shuffle and Sort

After the Map phase, Hadoop performs shuffle and sort.

All values having the same key are grouped together.

The grouped data is then sent to the reducers.

5. Reduce Phase

The Reducer adds the values for each key.

The final output is stored in HDFS.

Task Distribution

Since only a limited number of nodes are available, tasks should be distributed carefully.

If four worker nodes are available, different mapper tasks can be distributed among them.

The scheduler tries to execute a task on the node where the required HDFS block is already present. This is called data locality.

Data locality reduces network communication and improves performance.

How to Complete Within 20 Minutes?

The following methods can be used:

1. Parallel processing

Multiple mapper tasks should execute simultaneously.

2. Proper input splitting

Large log files should be divided into suitable blocks.

3. Use of Combiner

Combiner reduces network traffic during shuffle.

4. Data locality

Processing data close to where it is stored reduces data transfer.

5. Proper reducer allocation

Too few reducers may increase processing time, while too many reducers may create unnecessary overhead.

6. Avoid unnecessary processing

Only required log fields should be processed.

3.A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

YARN Scheduler Policy for ETL, Reporting and Machine Learning

Introduction

YARN stands for Yet Another Resource Negotiator. It is the resource management and job scheduling component of Hadoop.

YARN manages resources such as:

- CPU
- Memory
- Containers

In this problem, the same cluster is used for:

1. ETL jobs
2. Reporting jobs
3. Machine learning jobs

The scheduler should provide fairness, give priority to ETL jobs and recover when a node fails.

Need for Scheduling

Different jobs have different requirements.

For example:

- ETL may be time-sensitive.
- Reporting may need regular resources.
- Machine learning may require large CPU and memory resources.

If there is no proper scheduler, one large ML job could consume most of the resources and prevent other jobs from running.

Therefore, queues and priorities should be used.

1. ETL Queue

ETL should have the highest priority.

ETL means Extract, Transform and Load. ETL jobs prepare data for other applications.

Therefore, delays in ETL can affect reporting and analytics.

ETL can be given a higher percentage of cluster resources and higher scheduling priority.

2. Reporting Queue

Reporting jobs can be placed in a medium-priority queue.

Reports should receive sufficient resources but should not completely block ETL operations.

3. Machine Learning Queue

Machine learning jobs can require large amounts of CPU and memory.

A separate queue can be created for ML jobs so that they do not consume all the resources of the cluster.

Fairness

Although ETL has higher priority, the scheduler should still maintain fairness.

Fairness means that reporting and ML jobs should also receive resources.

Resource limits can be defined for each queue.

For example:

Queue	Priority	Resource Allocation
ETL	High	High
Reporting	Medium	Moderate
ML	Medium	Moderate

The exact percentages can be adjusted according to the organization's workload.

Node Failure Recovery

Node failure is possible in a Hadoop cluster.

If a worker node fails while running a task, YARN detects the failure using communication between the Resource Manager and Node Manager.

The failed task can then be restarted on another available node.

The process is:

Node Failure → Failure Detection → Task Rescheduling → New Node → Task Continues

This provides recovery from hardware or node failures.

Advantages

The proposed policy provides:

- ETL priority
- Fair resource sharing
- Better cluster utilization

- Protection from resource starvation
- Recovery from node failure
- Separation of different workloads

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Enterprise Data Ingestion from SAN, NAS and Operational Databases

Introduction

In an enterprise, data is usually stored in different systems. Some data may be stored in SAN, some in NAS, and some in operational databases.

SAN stands for Storage Area Network, while NAS stands for Network Attached Storage.

The enterprise wants to bring all this data into a big data platform. At the same time, it needs reliable backup and minimum duplication.

Proposed Data Ingestion Architecture

A data ingestion tool such as Apache NiFi or an ETL tool can be used between the data sources and the Hadoop platform.

The general flow is:

SAN + NAS + Databases → NiFi/ETL → Validation → Deduplication → HDFS

1. SAN Data

SAN provides block-level storage and is commonly used for enterprise applications.

Only the required data should be selected from SAN and transferred to the big data platform.

Repeated copying of the same data should be avoided.

2. NAS Data

NAS generally contains files and folders.

The ingestion system can monitor the required NAS locations and identify new or modified files.

This prevents the same file from being copied repeatedly.

3. Operational Databases

Operational databases contain structured information such as:

- Customer records
- Employee information
- Sales records
- Transactions

Instead of copying the complete database every time, incremental data extraction can be used.

Only newly inserted or modified records are transferred.

Change Data Capture (CDC) is another method that can be used to identify database changes.

4. Data Validation

Before storing the data in Hadoop, the data should be checked.

Validation can include:

- File format checking
- Schema checking
- Missing value checking
- Data type checking
- Invalid record detection

This improves data quality.

5. Deduplication

Duplication means storing the same data more than once.

For files, a **checksum or hash value** can be used.

If two files have the same checksum, they may contain the same content.

For database records, unique IDs or primary keys can be used to identify duplicate records.

This reduces unnecessary storage usage.

6. HDFS Storage

After validation and deduplication, the data can be stored in HDFS.

The data can be organized into:

- Raw data
- Processed data
- Curated data

The raw layer contains original data. The processed layer contains cleaned data, while the curated layer contains data prepared for analysis.

7. Backup Reliability

HDFS replication provides protection against DataNode failures.

However, for important enterprise information, an independent backup or disaster-recovery system should also be used.

Important metadata and configuration information should also be backed up.

Advantages

This architecture provides:

- Centralized data ingestion
- Reduced duplication
- Reliable backup
- Scalable storage
- Better data quality

5.Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache Ni Fi or ETL pipelines. Explain how your design satisfies all constraints.

End-to-End Hadoop Ecosystem Solution

Introduction

A Hadoop ecosystem contains several tools that work together to store, process, transfer and analyze large amounts of data.

An enterprise Hadoop solution should satisfy three major requirements:

1. Secure data movement
2. High availability
3. Integration with Apache Ni Fi or ETL pipelines

A suitable ecosystem can use Apache Ni Fi, HDFS, YARN, MapReduce, Spark and Hive along with security mechanisms.

1. Apache Ni Fi

Apache Ni Fi can be used for collecting data from different sources.

Sources may include:

- Databases
- Files
- Applications
- APIs
- Enterprise systems

Ni Fi can perform:

- Data collection
- Routing
- Filtering
- Transformation
- Validation
- Monitoring

Therefore, Ni Fi acts as an important entry point into the Hadoop ecosystem.

2. Secure Data Movement

Data should be protected while moving from one system to another.

TLS/SSL can be used to encrypt data during transmission.

Kerberos can be used for authentication within Hadoop.

Authentication ensures that only authorized users and services can communicate with Hadoop components.

Authorization mechanisms can also be used to control which users can access specific files and datasets.

3. HDFS

HDFS is used as the main distributed storage system.

It divides large files into blocks and distributes those blocks across Data Nodes.

Replication provides fault tolerance.

If one Data Node fails, another Data Node can provide the required replica.

4. HDFS High Availability

Name Node High Availability can be used to avoid a single point of failure.

There can be:

- Active Name Node
- Standby Name Node

The Standby Name Node is ready to take over if the Active Name Node fails.

This improves system availability.

5. YARN

YARN manages the resources required by applications.

It manages:

- CPU
- Memory
- Containers
- Application scheduling

Applications such as Spark and MapReduce can run using YARN.

YARN also helps distribute resources among different users and applications.

6. MapReduce

MapReduce is suitable for large-scale batch processing.

It contains:

- Mapper
- Shuffle and Sort
- Reducer

It can process huge datasets by dividing the workload into smaller tasks and executing them in parallel.

7. Apache Spark

Spark can be used for:

- Fast data processing
- Data analytics
- ETL
- Machine learning

Spark can run on YARN and use Hadoop storage.

8. Hive

Hive provides an SQL-like interface for querying large datasets.

Users who know SQL can use Hive to analyze Hadoop data without writing complex MapReduce programs.

9. High Availability

High availability can be achieved using:

- Name Node HA
- Multiple Data Nodes
- HDFS replication
- Resource Manager HA where required
- Multiple NiFi nodes
- Failure recovery

If one component fails, another component can continue the operation where redundancy has been configured.

10. Complete Data Flow

The complete process can be understood as:

Data Sources → Apache NiFi → Secure Transfer → HDFS → YARN → Spark/MapReduce/Hive → Analysis

For example, customer data can be collected through NiFi, securely transferred to Hadoop, stored in HDFS, processed using Spark, and finally queried using Hive.

Constraint Satisfaction

Secure data movement:

TLS/SSL protects data during transfer and Kerberos provides authentication.

High availability:

Name Node HA, replication and multiple nodes provide fault tolerance.

NiFi/ETL integration:

NiFi collects, transforms and routes data before storing it in Hadoop.

Scalable storage:

HDFS allows data to be distributed across many Data Nodes.

Resource management:

YARN manages CPU and memory resources.

Data processing:

MapReduce and Spark process large datasets.

Data analysis:

Hive provides SQL-based analysis.